CSL236 · Introduction to AI and Machine Learning

End-Term Project Report

PhishGuard

An AI phishing-domain detector shipped as a production-style ML system

Team member	Roll number
Shivansh Saroha	24CSU192
<Member 2 name>	<roll no.>
<Member 3 name>	<roll no.>

Section: _____

Programme: B.Tech CSE, Semester 5

Institution: The NorthCap University, Gurugram

Submitted: November 2026

Repository: github.com/Shivansh22Saroha/Ai-phising-detector
Live app: ai-phising-detector-kgt4.onrender.com
API docs: ai-phising-detector-kgt4.onrender.com/docs

1. Problem and dataset

1.1 Problem statement

Phishing links imitate real brands (for example `paypal.com.account-verify.security-check.ru`) or hide on free hosting platforms. Blocklists only know links that were already reported, so a brand-new phishing domain passes until someone flags it. **PhishGuard** is a binary classifier that predicts whether a link is phishing (`is_phishing = 1`) or legitimate (`0`) from the structure of its domain name, before the user clicks it. The model is served by a web app that also explains, through rule-based tips, what looks risky in the link.

1.2 Dataset

We use the **PhiUSIIL Phishing URL dataset** (Prasad & Chandra, *Computers & Security*, 2024; UCI Machine Learning Repository id 967; licence CC BY 4.0). It contains **235,795 URLs**: 134,850 legitimate (57%) and 100,945 phishing (43%). The CSV is 54 MB, above the 50 MB repository limit, so it is not committed: `data/download_data.py` downloads it, keeps only the `URL` and `label` columns, and flips the label (UCI uses 1 = legitimate) so that 1 means phishing.

Why PhiUSIIL instead of the mid-term dataset. The mid-term used the UCI Phishing Websites dataset (2012, 11,055 rows, 30 features). Its features are pre-computed codes (−1/0/1) for properties such as SSL state, domain age, web traffic rank, PageRank and Google indexing. It contains no raw URLs, so a deployed app cannot compute those features from a link a user types, and several depend on services that no longer exist (Alexa rank, Google PageRank). PhiUSIIL provides the raw URL for every row, is 21 times larger and much more recent, which lets us engineer our own features and compute exactly the same features at prediction time.

1.3 EDA highlights

Before modelling we checked whether a simple pattern in the URL already gives away the label (a *shortcut* that a model would learn instead of real phishing signals). It does:

Pattern in the raw URL	URLs	Share phishing
Has a path or query after the domain	28,684	100.0%
Ends with “/”	36,813	100.0%
Starts with http:// (not https)	51,749	100.0%
Starts with https://www.	137,299	1.8%

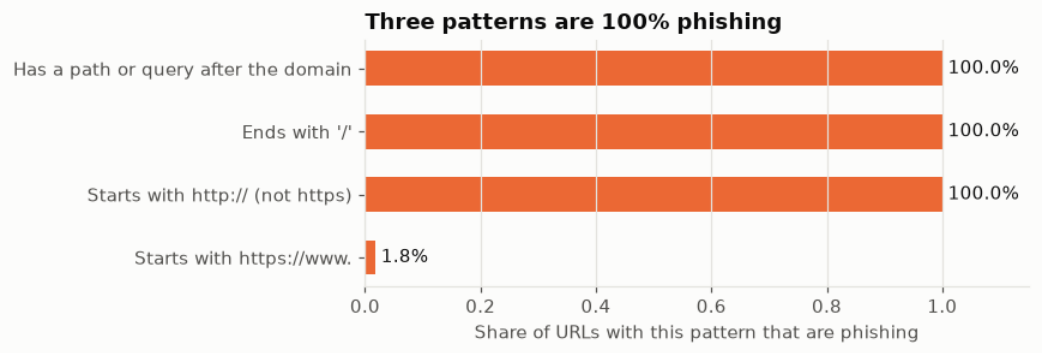


Figure 1. Three URL patterns are 100% phishing in PhiUSIIL

Legitimate rows are almost all bare `https://www.<domain>` homepages. A model trained on full URLs would learn “has a path = phishing” and flag real links such as `https://github.com/anthropics`. We therefore **normalise every URL to its hostname** (lower-cased, `www.` removed) and let the model see only that. Signals in the rest of the URL (`@`, raw IPs, `http`) are handled by rule-based tips in the app.

Many URLs share a host, so we **deduplicate to one row per host** (235,795 URLs → 219,094 hosts) and drop 58 hosts labelled both ways, leaving **219,036 hosts (38.5% phishing)**. This prevents the same host appearing in both the training and the test set.

1.4 Feature engineering and data contract

`features.py` turns each hostname into 16 numbers. The same file is imported by `train.py` and `app.py`, so training and serving can never compute features differently.

Feature	Type	Allowed range	Why it matters
host_length	int	1 – 253	Long hosts pack in brand names; 253 is the DNS limit
num_subdomains	int	0 – 125	`paypal.com.evil.ru` puts the brand in front of the real domain
num_hyphens	int	0 – 252	Hyphenated look-alikes (`pay-pal-secure`)
num_digits / digit_ratio	int / float	0 – 253 / 0 – 1	Random IDs and `amaz0n`-style swaps
is_ip	int	{0, 1}	Real services don't send users to bare IPs
host_entropy	float	0 – 6	Machine-generated random names score high
suspicious_keyword_count	int	0 – 13	Bait words: login, verify, secure, bank …
tld_length / is_risky_tld	int	0 – 63 / {0, 1}	Cheap TLDs abused for phishing (.xyz, .top …)
has_punycode	int	{0, 1}	Homograph attacks with non-Latin look-alike letters
longest_label_length	int	1 – 253	A long junk label hides a brand
is_free_hosting	int	{0, 1}	Free site builders cost attackers nothing
vowel_ratio / max_consonant_run	float / int	0 – 1 / 0 – 253	Random strings such as `xkqzvbt`
digit_letter_switches	int	0 – 252	IDs like `caseid1008934`
is_phishing (target)	binary	{0, 1}	Label

Every row of this contract is enforced by the Pydantic schema `PhishFeatures` in `app.py`: strict types (a string `"12"` or `true` is rejected for an integer), the ranges above, no unknown fields, no NaN or Infinity, and cross-field consistency checks (for example `num_digits ≤ host_length`). Any violation returns **HTTP 422** before the model is called.

2. System architecture

The system follows the reference workflow data → training → MLflow → model → API → Docker → CI → cloud. Figure 2 shows each stage with the file that implements it, the monitoring loop, and the path of a single prediction request.

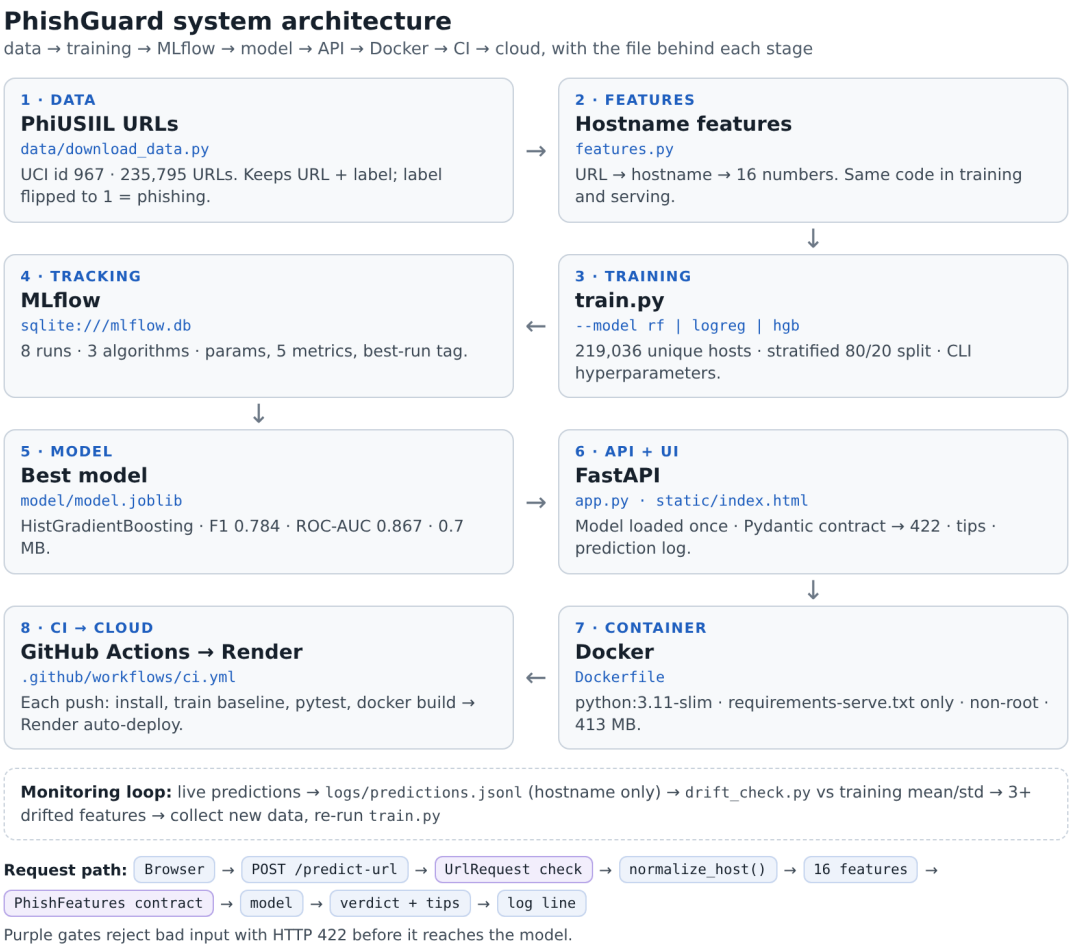


Figure 2. PhishGuard architecture, monitoring loop and request path

Layer	Technology
Language and data	Python 3.11, pandas, NumPy
Modelling	scikit-learn 1.9 (Logistic Regression, Random Forest, HistGradientBoosting), joblib
Experiment tracking	MLflow 3.18 with SQLite backend (`mlflow.db`)
API and validation	FastAPI 0.162, Pydantic 2.14, Uvicorn; HTML/CSS/JS UI in `static/index.html`
Testing	pytest 9.2 with FastAPI TestClient
Container	Docker, `python:3.11-slim`, `requirements-serve.txt`
CI/CD and hosting	GitHub Actions; Render free tier (Docker runtime, auto-deploy, health check)

3. Experiments and model comparison

`train.py` takes the algorithm and its hyperparameters as command-line flags (`--model`, `--n_estimators`, `--max_depth`, `--C`, `--learning_rate`, `--max_iter` …), splits the 219,036 hosts 80/20 with stratification (`random_state=42`), and logs parameters and five test-set metrics to MLflow. All models use `class_weight="balanced"`. The model file is replaced only when a run beats the saved model's F1, so `model/model.joblib` is always the best run.

Run	Algorithm	Hyperparameters	Acc.	Prec.	Recall	F1	AUC
1	Logistic Regression	C = 1.0	0.841	0.879	0.680	0.767	0.845
2	Logistic Regression	C = 0.1	0.842	0.881	0.680	0.768	0.846
3	Random Forest	100 trees, depth 6	0.841	0.882	0.677	0.766	0.850
4	Random Forest	200 trees, depth 12	0.853	0.904	0.690	0.782	0.863
5	Random Forest	300 trees, depth 20, leaf 2	0.852	0.896	0.697	0.784	0.866
6	HistGradientBoosting	lr 0.1, 300 iter	0.852	0.891	0.700	0.784	0.867
7	HistGradientBoosting	lr 0.05, 600 iter	0.852	0.892	0.700	0.784	0.867

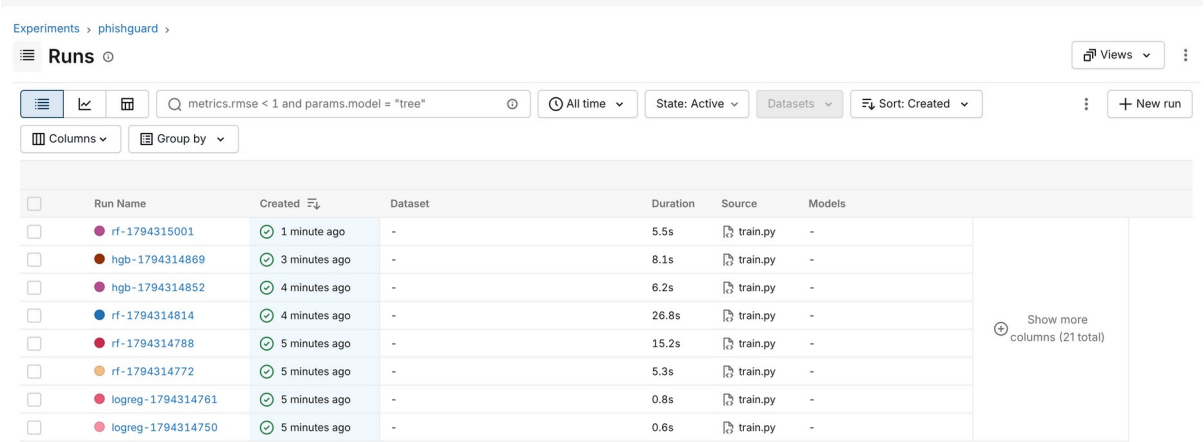


Figure 3. MLflow UI listing the 8 tracked runs (Random Forest, Gradient Boosting, Logistic Regression)

3.1 Model choice

We deploy **HistGradientBoosting (learning rate 0.05, 600 iterations)**. It has the best F1 (0.784) and ROC-AUC (0.867). The deepest Random Forest ties on F1 but trains about twice as slowly, and the boosting model file is only 0.7 MB, which keeps the container small and CI fast. Logistic Regression is lower on every metric, which shows that the relationship between features and label is non-linear. We select on **F1** rather than accuracy because the classes are imbalanced and both missed phishing (recall) and false alarms (precision) matter.

A feature-engineering experiment also mattered: adding `is_risky_tld`, `vowel_ratio`, `max_consonant_run` and `digit_letter_switches` raised the boosting model's F1 from 0.743 to 0.784 and ROC-AUC from 0.846 to 0.867.

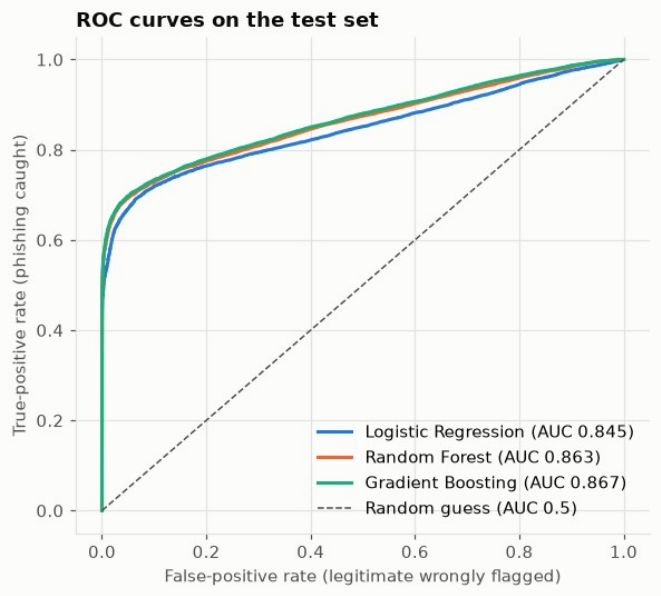


Figure 4. ROC curves on the 43,808-host test set

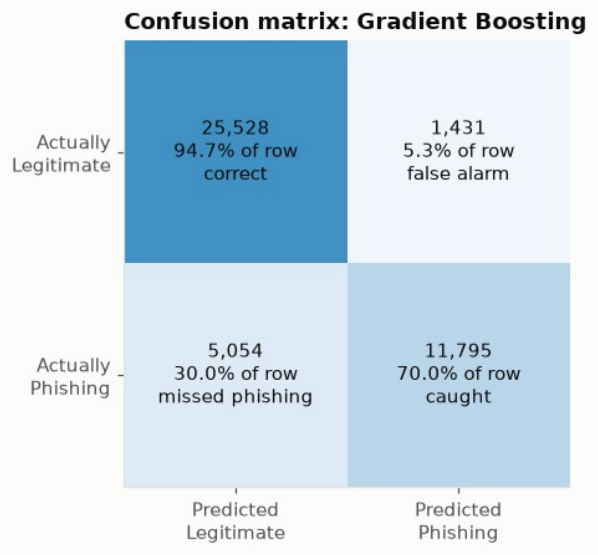


Figure 5. Confusion matrix of the deployed model at threshold 0.5

The deployed model passes 94.7% of legitimate hosts (5.3% false alarms) and catches 70.0% of phishing hosts. **Permutation importance** (Figure 6) shows that it relies most on host length, the risky-TLD flag and the number of subdomains. Features that repeat information already present (for example `digit_ratio`, which is `num_digits ÷ host_length`) score close to zero.

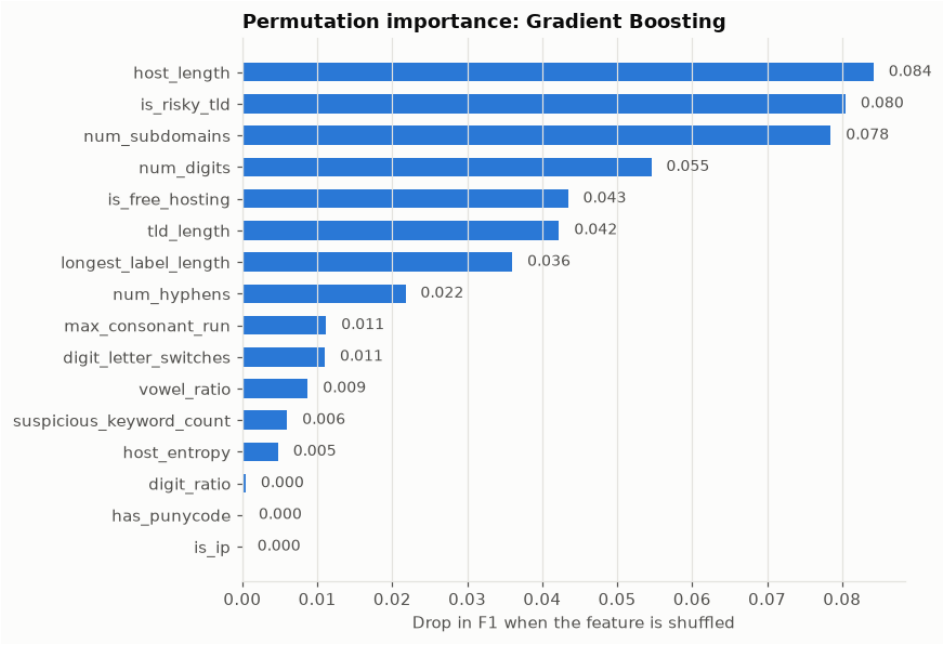


Figure 6. Permutation importance (drop in F1 when a feature is shuffled)

F1 is almost flat between thresholds 0.40 and 0.60 (0.778–0.785), so the default 0.5 already sits at the F1 peak. Lowering the threshold to 0.35 would catch about four more phishing hosts in a hundred but cost about ten points of precision.

4. Testing, CI and deployment

4.1 API and UI

`app.py` loads the model **once at startup** (`MODEL = joblib.load(...)` at module level) and exposes `GET /health`, `POST /predict` (the 16 contract features), `POST /predict-url` (a raw URL, converted to features server-side and validated by the same contract), the UI at `/`, and Swagger documentation at `/docs`. The UI shows the verdict, the phishing probability and 2–3 rule-based tips about things the model deliberately does not see: `http` instead of `https`, an `@` in the link, a raw IP address, punycode, free hosting, bait words, many subdomains, a risky TLD or a very long link.

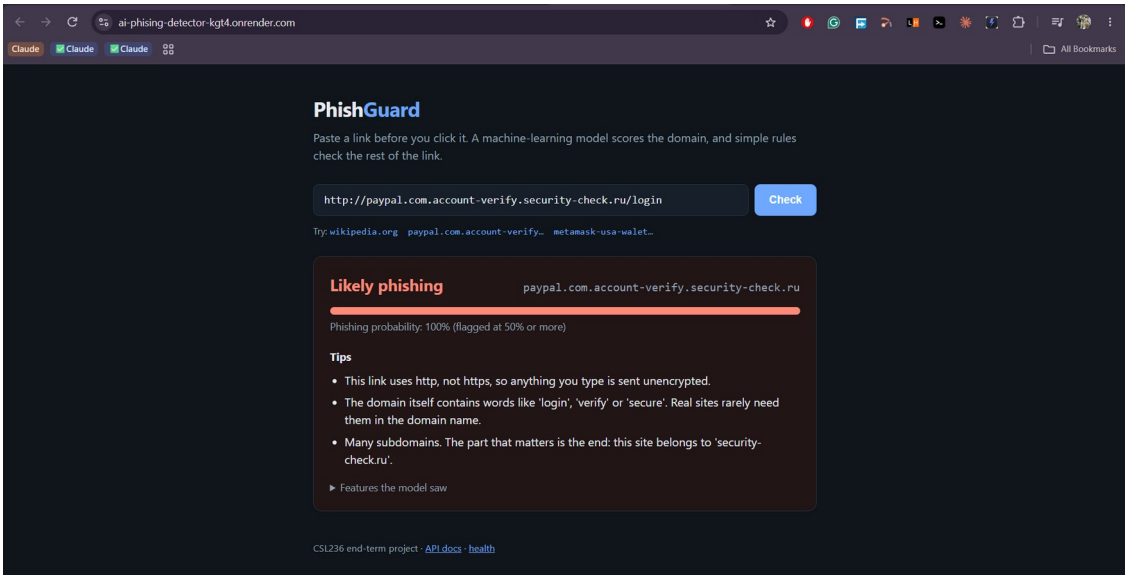


Figure 7. The live deployment flagging a fake PayPal link with three tips

4.2 Automated tests

`tests/test_app.py` contains **22 passing tests**:

- health check, UI loads, and HEAD requests allowed on `/` and `/health` (Render probes with HEAD);
- **domain logic:** a phishy host (`secure-login-verify-account8842.firebaseapp.com`) must score higher than `google.com`, and adding a path to a URL must not change its score (guards against the dataset shortcut);
- **boundary and contract tests expecting 422:** out-of-range values, wrong types (`"12"`, `12.5`, `true`), missing and unknown fields, **NaN**, impossible feature combinations, and invalid URLs;
- `/predict-url` returns 2–3 tips and writes a log line that contains the hostname but never the URL path.

The NaN test found a real bug: Pydantic rejected NaN correctly, but FastAPI's default 422 response echoes the input back and JSON cannot encode NaN, so the client received a 500. A custom validation handler now makes the response safe.

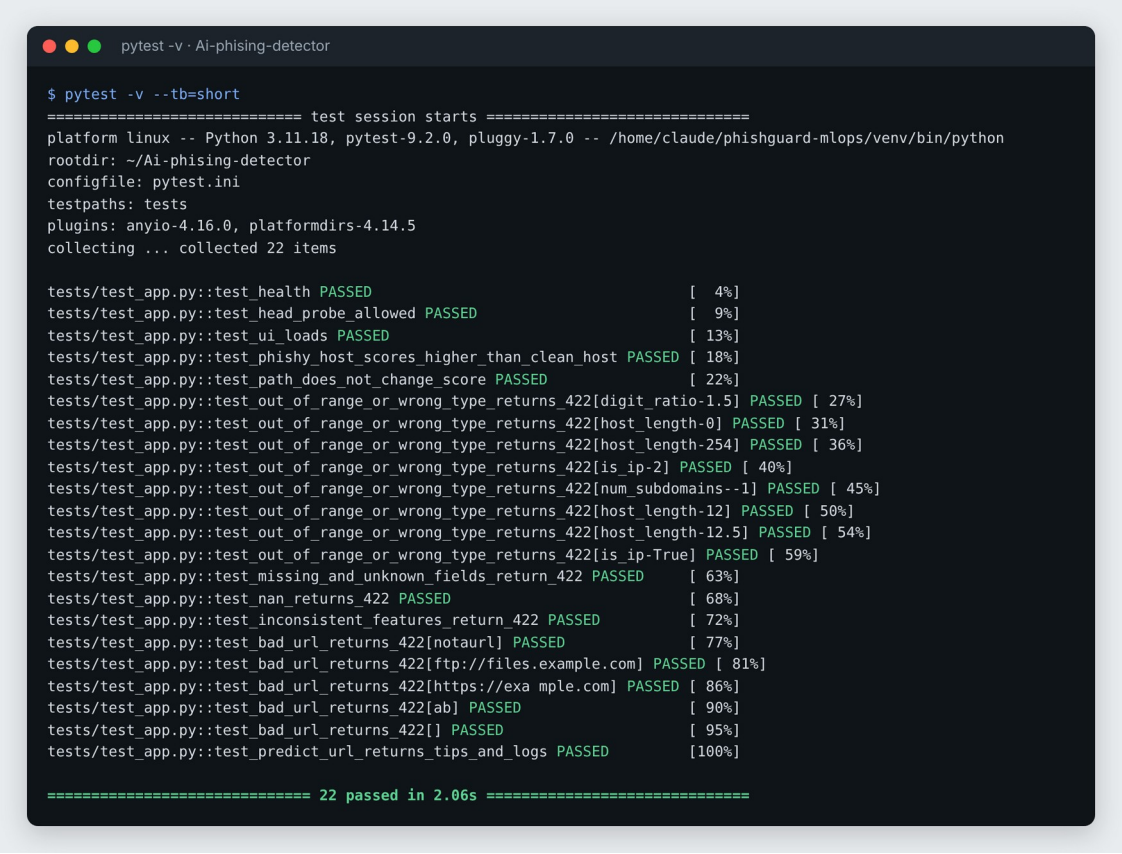


Figure 8. pytest -v output: 22 passed

4.3 Docker

The image builds from `python:3.11-slim` and installs only `requirements-serve.txt` (NumPy, scikit-learn, joblib, FastAPI, Uvicorn, Pydantic). MLflow, pandas and pytest are left out because the server only loads a model and answers requests. scikit-learn is pinned to the same version in both requirement files so the saved model loads exactly as trained. The container runs as a non-root user, has a `HEALTHCHECK` on `/health`, listens on port 8000 (or the platform's `$PORT`) and measures **413 MB**.

4.4 Continuous integration

`.github/workflows/ci.yml` runs on every push and pull request to `main`. The first job installs `requirements.txt`, downloads a 40,000-row stratified sample, trains a baseline Random Forest and

runs pytest. The second builds the Docker image, starts it and polls `/health`. The third, optional job triggers a Render deploy hook. The first push failed because of a YAML syntax error in the workflow, which was fixed; every run since has been green.

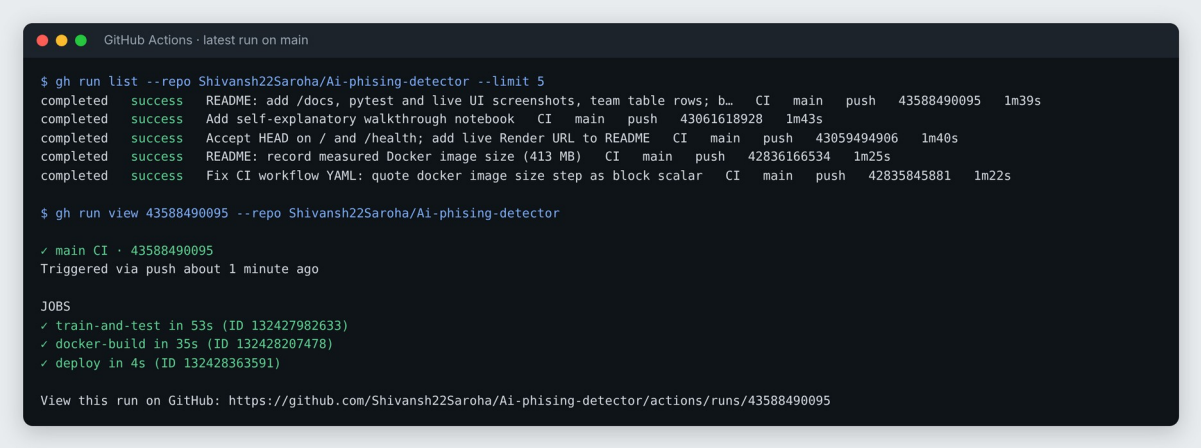


Figure 9. Latest GitHub Actions runs on main (GitHub CLI view)

4.5 Deployment and monitoring

The container runs on **Render's free tier** at ai-phising-detector-kgt4.onrender.com. Render checks `/health`, restarts the container if it fails, and redeploys automatically on every push to `main`. Every prediction is appended to `logs/predictions.jsonl` with a timestamp, the hostname, the 16 features, the probability and the verdict; the full URL is never logged because paths and query strings can contain tokens. `drift_check.py` compares the logged feature means with the training mean and standard deviation saved by `train.py` and recommends retraining when three or more features drift by more than one standard deviation.

5. Limitations and next steps

- **Hostname only.** Phishing pages hosted on a hacked legitimate website are invisible to the model. Many missed phishing hosts in the test set are short university and country domains, which look like exactly this case. Next: add page-content features and domain age from WHOIS.
- **Legitimate data is homepages of registered domains,** so legitimate login subdomains look unusual; `login.microsoftonline.com` is a known false positive. Next: add legitimate subdomain URLs and an allowlist of top domains.
- **Recall is 0.70.** Some phishing domains look completely ordinary. Next: tune the threshold on a separate validation set and add character n-gram features.
- **Dataset age.** PhiUSIIL was published in 2024, and attack patterns change. Next: evaluate on a newer dataset and retrain when the drift check fires.
- **Free hosting.** The Render instance sleeps after about 15 minutes idle, so the first request can take around 50 seconds.
- **Approximate subdomain count.** Multi-part suffixes such as `.co.uk` are over-counted. Next: use the Public Suffix List.

6. Individual contribution

Member	Roll number	Contribution
Shivansh Saroha	24CSU192	<fill in>
<Member 2 name>	<roll no.>	<fill in>

Member	Roll number	Contribution
<Member 3 name>	<roll no.>	<fill in>

Work areas: dataset choice and EDA; feature engineering (`features.py`); training and MLflow experiments (`train.py`); FastAPI service, data contract and UI; automated tests; Docker image; CI/CD workflow; Render deployment, logging and drift check; README, report and presentation.

References

[1] Prasad, A. and Chandra, S. (2024). PhiUSIIL: A diverse security profile empowered phishing URL detection framework based on similarity index and incremental learning. *Computers & Security*, 136, 103545. Dataset: UCI Machine Learning Repository, id 967.

[2] Mohammad, R., Thabtah, F. and McCluskey, L. (2015). Phishing Websites dataset. UCI Machine Learning Repository, id 327.

[3] Pedregosa, F. et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.

[4] FastAPI, Pydantic, MLflow, Docker, GitHub Actions and Render official documentation.